

IronFit Movement Mirror — Concept, Brief & Build Rationale

A companion document to the Trainer Guide: what we set out to build, why, the thinking and domain research behind it, and how the prototype came together.

The original brief

Reconstructed from the project's founding requirements and design notes — the substance of what was asked for. (Exact original wording can be substituted in.)

Build a **mobile-first web app that helps IronFit clients rehearse Functional Range Systems (FRS) movements between sessions**. The trainer (Lee, FRC/FRA certified) records reference videos of each movement; the app extracts joint-angle data from those videos and then gives clients **live, joint-by-joint feedback** while they perform the same movement in front of their phone camera.

The guiding constraints in the brief were explicit and unusually principled:

- **"It's a mirror with a memory, not a coach."** The trainer defines what correct looks like; the app helps the client rehearse it accurately. The app must not invent movements, prescribe loads, or progress anyone — the trainer does that.
- **Privacy first.** No client video or pose data should ever leave the device.
- **Honesty over hype.** Be explicit about what the technology can and can't do, flag low-confidence readings rather than fake precision, and *"tell me what will break before I have to find out."*
- **Real performance targets** on real phones (hold ~24fps live; warn below 15).
- **A coaching tool, not a medical device** — it does not diagnose injury or replace in-person assessment.

This document explains the reasoning that turned that brief into the working prototype.

The problem it solves

FRS training (CARs, PAILS/RAILs, controlled articular work) lives or dies on **precision between sessions**. A client sees their trainer occasionally, then goes home and rehearses — often drifting from the exact range and control the trainer prescribed, with no feedback until the next session weeks later. Cues get forgotten; compensations creep in; progress stalls and nobody sees why.

Generic fitness apps don't help here because they optimize for the wrong thing: streaks, points, calories, and rep counts. FRS isn't about *how many* — it's about *how well*: did the joint actually reach end range, was it controlled, did the body cheat the movement with a compensation. The brief demanded a tool built around **range and control**, measured against the trainer's own standard.

Domain grounding & research

The design is grounded in how FRS is actually coached, and in an honest read of what consumer phone-based pose estimation can and cannot measure.

FRS coaching concepts the app models directly

- **Movement types** — CAR (controlled articular rotation), PAILS/RAILs, flows.
- **Range of motion (ROM)** measured per joint against a clean reference rep.

- **Compensation patterns** — the specific ways a body cheats a movement (e.g. lumbar substitution, scapular hiking), each paired with a coaching cue.
- **The trainer's reference is the standard.** There is no "ideal" baked in; the app compares the client to the trainer's own recorded range, not a textbook.

Honest limits of 2D pose estimation (analysis done up front)

A single phone camera with a 2D pose model (MediaPipe) sees joint *positions*, not twist around a limb's long axis. Rather than paper over this, we mapped exactly where it would be weak and built the honesty in:

- **Rotation-dominant movements** (shoulder/hip internal-external rotation, forearm pronation/supination) are flagged **approximate** and never shown as a confident reading; the app nudges the user to also film from the side.
- **Ankle / foot-dependent ranges** are capped at "reduced" confidence — feet are the noisiest landmarks.
- **Thoracic vs lumbar spine** can't be separated by a single-camera proxy, so a spinal flag means "the trunk moved," not a segment-specific claim.
- Every joint carries a **confidence badge**; color is never the only signal.

Thresholds: defensible defaults, not invented standards

FRS does not publish numeric app tolerances. So every threshold (what counts as "green" vs "short," what drift counts as a compensation, rep-detection sensitivity) is a **documented, tunable default for the trainer to review** — centralized in one file with its reasoning and a confidence rating, explicitly flagging the ones most in need of real-world tuning. We did not dress guesses up as science.

Key design decisions & the reasoning behind each

- 1. On-device processing, privacy by architecture.** The same pose model and angle math run in the browser on both sides. Client video is analyzed on their phone and discarded; only the *scores* (ROM %, compensation flags, a confidence rating) are stored. This isn't a setting that can be misconfigured — there is no server-side video pipeline to leak.
- 2. "Mirror with a memory," not an AI coach.** The app deliberately does not generate programs or progress clients. It rehearses what the trainer prescribed. This keeps the trainer central, keeps the scope honest, and avoids the failure mode of an app confidently coaching something it can't actually assess.
- 3. Baseline + running-peak comparison, no tempo-syncing.** Time-syncing a client to the trainer's exact tempo is brittle. Instead each joint locks a baseline on the first good frame, tracks the running peak, and compares *range achieved* to the reference range. A client moving at half speed still gets honest end-range feedback — and the summary says so out loud.
- 4. Non-vanity coaching metrics.** The dashboards answer coaching questions, not engagement ones. The system surfaces four signals worth a trainer's attention: a **recurring compensation, range that's stalled or declining**, a client who's **gone quiet**, and a **camera setup that's hurting feedback quality**. No streaks, points, or totals-for-their-own- sake.
- 5. Intake-link client assignment.** The earliest version could only attach a client to a trainer by hand (a SQL update) — a known gap. The prototype replaces it: each trainer has a unique intake link; a client fills a short intake form and is **auto-assigned to that trainer**, with their goals and injury history captured up front. This also fixed a real bug where every new client defaulted to a single trainer.

6. Library + "Your focus." Clients see the full movement library, but trainers can pin a few movements as that client's focus (with a note), highlighted at the top. It balances autonomy with direction.

7. Practice-wide overview shared by all trainers. For a small multi-trainer practice, all trainers share one overview of every roster, who needs attention, and the ability to reassign clients — chosen over a rigid owner-only admin tier to match how a small team actually operates.

8. Scanner-proof authentication. Auth evolved deliberately: magic link !' 6-digit email code !' finally **email + password with no email in the login path at all**, so corporate email scanners, rate limits, and delivery failures can't block sign-in.

How it was built — iteration timeline

1. Core libraries first. Pose detection, reference extraction, and live comparison were built as pure, unit-tested modules before any UI.

2. The app shell. Trainer and client flows, Supabase wiring, row-level security, and the live performance overlays.

3. Honest documentation. A developer + trainer guide, the recording guide, and a frank "what I expect to break" section.

4. Production hardening. Self-contained build, connected to its own database, and the authentication rework for reliable sign-in.

5. The coaching layer. Trainer and client dashboards with the non-vanity metrics, plus in-app realtime messaging.

6. The operating model. Intake-link onboarding, per-client programs, the practice-wide overview, and the trainer guide — turning a single-trainer demo into a real multi-trainer practice tool.

Technology & architecture

- **Frontend:** React + Vite (TypeScript), mobile-first, deployed on Vercel.
- **Pose & analysis:** MediaPipe Tasks Vision, running entirely in the browser. Reference extraction (trainer) and live comparison (client) share identical angle math, so they're directly comparable — and there is no server-side ML.
- **Backend:** Supabase (Postgres) with row-level security enforcing that clients see only their own data and trainers see their practice. Private storage buckets for reference videos with signed playback URLs.
- **Quality:** a unit-test suite over the geometry, extraction, and comparison logic, plus end-to-end smoke tests.

A deliberate scalability note was built in: in-browser extraction is right for now, but the extractor core is pure and decoupled, so it can be lifted into a serverless worker the moment the trainer is processing many or long clips.

Status & roadmap

The prototype is a **working multi-trainer practice tool**: trainers build movements, recruit and onboard clients via intake links, program each client's focus, coach from the dashboards, message clients, and share a practice-wide overview. Clients rehearse with live feedback and watch their range improve.

Intentionally deferred to last, per plan:

- **Billing** — kept out until the rest of the app is settled; clients currently see a neutral "handled by your trainer" placeholder.

- **Known follow-ups** documented honestly: audio cues (text-to-speech of the trainer's cues), auto-generated captions, per-rep breakdowns, left/right symmetry scoring, and moving reference extraction to a worker as volume grows.

The whole thing is built to grow: principled scope, honest limits, and a clean separation between what the technology measures and what the trainer decides.